If the `EmailSender` class is as simple as the previous example and has no dependencies, you may not see any need to adopt a different approach to creating objects. And to an extent, you'd be right. But what if you later update your implementation of `EmailSender` so that some of the email-sending logic is implemented by a different class?

Currently, `EmailSender` would need to do many things to send an email. It would need to

- Create an email message.
- Configure the settings of the email server.
- Send the email to the email server.

Doing all that in one class would go against the single-responsibility principle (SRP), so you'd likely end up with `EmailSender` depending on other services. Figure 8.1 shows how this web of dependencies might look. `RegisterUser` wants to send an email using `EmailSender`, but to do so, it also needs to create the `MessageFactory`, `NetworkClient`, and `EmailServerSettings` objects that `EmailSender` depends on.